

Comparison of Transfer Learning and Random Initialization for Image Classification

Shriman Raghav Srinivasan

Khoury College of Computer Sciences, Northeastern University, Boston, MA

Email: srinivasan.shri@northeastern.edu

Abstract—This paper presents a controlled, empirical comparison of transfer learning and training from scratch using the ResNet-18 architecture on the six-class Intel Image Classification dataset. To isolate the effects of feature pretraining quality from those of trainable capacity, I define a capacity-matched design featuring three training conditions: (i) *pretrained-frozen*, where ImageNet-pretrained weights are loaded and early layers are frozen; (ii) *random-frozen*, where early layers are randomly initialized and frozen as a capacity-matched control; and (iii) *random-full*, where the entire network is trained from scratch. In all frozen conditions, batch normalization layers in the frozen block are kept in evaluation mode to prevent representation drift. The models are trained with validation-based early stopping across three distinct random seeds. Evaluation on the held-out test split demonstrates that the *pretrained-frozen* model achieves a mean test accuracy of 93.27%, significantly outperforming the *random-frozen* (66.54%) and *random-full* (84.69%) configurations. The results isolate a substantial accuracy benefit ($> 50\%$) directly attributable to the quality of pretrained features under matched capacities. A freeze-depth ablation further shows that a linear probe on the frozen pretrained features already attains most of this accuracy, which shows how much pretraining matters when data and compute are limited.

Index Terms—Transfer Learning, Image Classification, Parameter Freezing, Network Capacity, Empirical Analysis.

I. INTRODUCTION

Convolutional networks classify images well, but training one from random weights usually takes a large labeled dataset and a lot of compute to avoid overfitting and to converge at all. Transfer learning is the standard way around this: start the network from weights learned on a large source dataset such as ImageNet [1], then adapt it to a smaller target task.

A naive comparison between a pre-trained network with frozen early layers and a fully trained network from random initialization confounds two distinct factors: the quality of the pretrained representations, and the reduced trainable parameter count (trainable capacity) of the frozen network.

To resolve this ambiguity, this paper implements a three-condition experimental design to isolate these factors:

- 1) **Pretrained-Frozen (PT-F)**: ResNet-18 model initialized with ImageNet weights, where early layers (θ_f) are frozen, and only the final convolutional block (`layer4`) and the classification head (θ_t) are optimized.
- 2) **Random-Frozen (R-F)**: A capacity-matched control identical to PT-F, but with θ_f randomly initialized and frozen.
- 3) **Random-Full (R-FL)**: The entire ResNet-18 network is initialized randomly and fully trained.

Condition (i) against (ii) isolates pretraining quality at matched capacity, and (ii) against (iii) isolates capacity under random initialization. I run all conditions on the Intel Image Classification dataset across several seeds with validation-based early stopping, and compare final accuracy and convergence speed.

II. RELATED WORK

The utility of pre-trained visual representations and the limits of transfer learning have been studied extensively:

He et al. [2] introduced the Deep Residual Learning framework (ResNet), establishing that residual connections allow networks to be trained to much greater depths without optimization degradation. ResNet-18 serves as the standard, lightweight backbone in this work due to its computational feasibility and representative residual structure.

Yosinski et al. [3] investigated the transferability of features in deep neural networks. They proved that first-layer features are general-purpose (e.g., edge and color detectors) regardless of the dataset, while final-layer features are task-specific. Crucially, they observed that freezing early layers maintains general features, but splitting a network mid-way can cause optimization difficulties due to co-adaptation of features in adjacent layers. This motivates my partition boundary, where only the final stage (`layer4`) and head are trainable.

He et al. [4] challenged the necessity of ImageNet pre-training for target tasks in their seminal paper “Rethinking ImageNet Pre-training”. They demonstrated that models trained from scratch (random initialization) can match or even exceed the accuracy of pre-trained models on target tasks such as COCO object detection, provided they are trained for long enough. They concluded that pre-training primarily speeds up convergence rather than improving generalization limits when data is sufficient. My work explores whether these findings hold for scene classification under small data and tight training patience budgets.

A related line of work showed that pre-trained features transfer even *without* fine-tuning. Donahue et al. [5] (DeCAF) and Sharif Razavian et al. [6] found that a linear classifier on *frozen* activations from an ImageNet-pretrained CNN is a strong baseline across recognition tasks. That “off-the-shelf features” result is the reason my freeze-depth ablation includes a linear-probe endpoint, where only the head is trained on top of fully frozen features. Kornblith et al. [7] later compared transfer across many architectures and found that stronger

ImageNet models tend to transfer better, which points to representation quality rather than capacity as the driver of downstream accuracy. That is exactly the factor my capacity-matched control isolates.

Because the target task is scene classification, the source-domain gap is also relevant: Zhou et al. [8] showed with the Places database that scene-centric pre-training yields features better suited to scene recognition than object-centric ImageNet features. My study uses ImageNet initialization throughout, so the reported transfer benefit is a lower bound on what a scene-tuned source might provide.

III. METHODS

A. Base Architecture and Partitioning

I employ ResNet-18 [2] as the base architecture. Let f_θ denote the network with parameters θ . I partition θ into a frozen subset θ_f and a trainable subset θ_t . Following findings that early layers learn low-level, generic representations [3], I define θ_f as the input convolution (`conv1`), first batchnorm (`bn1`), and the first three residual stages (`layer1`, `layer2`, `layer3`). The trainable subset θ_t consists of the final residual stage (`layer4`) and the 6-class output layer (`fc`), representing the task-specific adapter.

In total, θ_f contains 2,782,784 frozen parameters, while θ_t contains 8,396,806 trainable parameters (including batchnorm layers in `layer4` and the classification linear weights and bias).

B. Batch Normalization Handling

Batch Normalization (BN) [9] layers keep track of running statistics (mean and variance) during training. If the early layers (θ_f) are frozen but their BN layers keep updating those statistics, the network’s internal representation drifts and training collapses. To prevent this, I enforce that all BN layers within θ_f are explicitly set to evaluation mode (`model.eval()`) during training. Thus, their running statistics are frozen and they function as deterministic linear transforms matching their initialization. This handling applies only to the two frozen conditions: in *random-full* there is no frozen subset, so all BN layers remain in training mode and update their running statistics normally. The BN treatment therefore differs across conditions by design, following each one’s freeze policy.

C. Data Preprocessing and Splitting

The Intel Image Classification dataset [10] contains natural scene images of size 150×150 pixels across 6 classes: *buildings*, *forest*, *glacier*, *mountain*, *sea*, and *street*. Images are preprocessed identically across all conditions:

- 1) Resized such that the shorter side is 224px.
- 2) Center-cropped to 224×224 pixels.
- 3) Normalized per-channel using ImageNet mean $\mu = [0.485, 0.456, 0.406]$ and standard deviation $\sigma = [0.229, 0.224, 0.225]$.

The dataset is partitioned as follows: the standard training split (`seg_train`) containing 14,034 images is divided into a

90% training subset (12,631 images) and a 10% validation subset (1,403 images) carved out using class-stratified sampling to preserve label distributions. The test split (`seg_test`) containing 3,000 images is held out completely and only used for final model evaluation.

IV. EXPERIMENTS AND RESULTS

A. Experimental Setup

For all three conditions, I optimize the trainable parameters θ_t to minimize the cross-entropy loss:

$$\mathcal{L}(\theta_t) = -\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(y_i | x_i) \quad (1)$$

Training uses SGD with momentum $\beta = 0.9$, weight decay $\lambda = 10^{-4}$, and a learning rate $\eta = 10^{-3}$. I set the batch size to 32.

To ensure fair comparison, validation loss is evaluated at the end of each epoch. I apply early stopping with a patience of 5 epochs: training terminates if validation loss fails to improve for 5 consecutive epochs, and the checkpoint with the lowest validation loss is saved. All experiments are executed across three seeds: 42, 100, and 2026.

B. Overall Test Performance

Table I summarizes the test accuracy (mean and standard deviation across seeds) and convergence speed (epochs to early stopping) for each condition.

For context against prior work, publicly reported baselines on the Intel Image Classification dataset [10] using ImageNet-pretrained CNN backbones typically fall in the low-to-mid 90% range on the held-out test split. My *pretrained-frozen* result (93.27%) is consistent with these community baselines despite freezing the entire early feature extractor and training only `layer4` and the head, indicating that the frozen-transfer configuration recovers most of the accuracy attainable by full fine-tuning on this task.

TABLE I
TEST ACCURACY AND CONVERGENCE SPEED COMPARISON. EPOCHS REPORTED AS MEAN $\pm$ STD ACROSS SEEDS.

Condition	Mean Test Acc.	Std. Dev.	Epochs
Pretrained-Frozen	93.27%	0.23%	8.0 ± 0.8
Random-Frozen	66.54%	4.04%	8.7 ± 2.1
Random-Full	84.69%	1.87%	15.0 ± 2.2

C. Detailed Class Metrics

For each condition, I identify the seed closest to the mean test accuracy (representative run) to report detailed class-wise metrics. These representative runs correspond to Seed 100 for PT-F, Seed 100 for R-F, and Seed 100 for R-FL.

Table II details the per-class precision, recall, and F1-score for the representative seed of each condition.

TABLE II
PER-CLASS TEST METRICS FOR REPRESENTATIVE RUNS

Class	Pretrained-Frozen			Random-Frozen			Random-Full		
	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1
Buildings	93.2%	93.4%	93.3%	53.0%	64.5%	58.2%	82.5%	82.8%	82.6%
Forest	98.7%	99.4%	99.1%	62.5%	97.9%	76.3%	96.4%	95.6%	96.0%
Glacier	93.9%	82.8%	88.0%	78.7%	53.5%	63.7%	89.2%	70.2%	78.5%
Mountain	86.3%	92.2%	89.1%	67.1%	69.5%	68.3%	76.4%	85.0%	80.4%
Sea	95.1%	98.6%	96.8%	76.4%	49.4%	60.0%	84.7%	88.2%	86.5%
Street	93.3%	94.4%	93.8%	68.1%	64.7%	66.3%	82.8%	89.4%	86.0%

D. Confusion Matrices

Figure-style confusion matrices for the representative (closest-to-mean) seed of each condition are given in Table III. Rows are the true class and columns the predicted class, with labels abbreviated as B (buildings), F (forest), G (glacier), M (mountain), Se (sea), and St (street).

TABLE III
TEST CONFUSION MATRICES FOR REPRESENTATIVE RUNS (ROWS: TRUE, COLUMNS: PREDICTED)

Pretrained-Frozen (Seed 100)						
	B	F	G	M	Se	St
B	408	0	0	0	1	28
F	0	471	0	1	0	2
G	1	4	458	71	16	3
M	1	2	28	484	9	1
Se	2	0	2	3	503	0
St	26	0	0	2	0	473
Random-Frozen (Seed 100)						
	B	F	G	M	Se	St
B	282	64	0	11	7	73
F	4	464	0	1	0	5
G	64	29	296	86	33	45
M	41	41	36	365	36	6
Se	89	26	43	77	252	23
St	52	118	1	4	2	324
Random-Full (Seed 100)						
	B	F	G	M	Se	St
B	362	3	2	1	6	63
F	2	453	0	4	2	13
G	6	5	388	111	36	7
M	12	3	30	446	32	2
Se	15	3	14	20	450	8
St	42	3	1	2	5	448

E. Training Dynamics

Figure 1 plots validation accuracy and loss per epoch for every run. The pretrained-frozen runs begin close to their final accuracy and stay essentially flat, converging within a few epochs. Random-full instead climbs steadily over 12–17 epochs toward a lower plateau, and random-frozen is unstable and low, with large validation-loss spikes that reflect the difficulty of mapping arbitrary frozen features onto class labels. This makes the convergence-speed advantage of pretraining visible: it is not only more accurate but also immediately usable.

F. Feature-Space Separability

To visualize *why* pretrained features help, Figure 2 projects the 512-dimensional penultimate features of the test images onto their first two principal components. The pretrained-frozen features form clearly separated per-class clusters, and the residual overlaps (glacier/mountain/sea, and buildings/street) mirror exactly the errors seen in the confusion matrix. The random-frozen features are largely entangled: the trainable head receives little linearly separable structure, which directly explains the large accuracy gap under matched capacity.

G. Freeze-Depth Ablation

The three main conditions fix a single freeze boundary. To test how sensitive the result is to that choice, I sweep the boundary on the pretrained model: training only `fc` (a linear probe on frozen ImageNet features), then progressively unfreezing `layer4`, `layer3`, `layer2`, and finally the whole network. To keep the sweep tractable this ablation uses a fixed 40% stratified subset of the training set at a single seed; the validation and test splits are unchanged, so accuracies are directly comparable within the sweep (and consequently sit slightly below the full-data numbers of Table I).

Figure 3 shows the result. A bare linear probe already reaches 91.3%, confirming that the frozen ImageNet features are highly linearly separable for this task. Unfreezing `layer4` adds about 1.6 points to 92.9%, after which accuracy is flat: unfreezing `layer3`, `layer2`, or the full network yields 92.8% each, while the trainable parameter count rises from roughly 3k to 11.2M with no accompanying benefit. This validates the partition used in the main experiments: training `layer4` plus the head captures essentially all of the attainable accuracy, and deeper fine-tuning only adds cost.

TABLE IV
FREEZE-DEPTH ABLATION (PRETRAINED RESNET-18, 40% TRAIN SUBSET, SEED 42)

Trainable from	Trainable Params	Test Acc.	Epochs
<code>fc</code> only (linear probe)	3,078	91.30%	28
<code>layer4</code> (main PT-F)	8.40M	92.87%	9
<code>layer3</code> → <code>fc</code>	10.50M	92.80%	9
<code>layer2</code> → <code>fc</code>	11.02M	92.77%	9
Full fine-tune	11.18M	92.77%	9

Table IV lists the corresponding numbers. Two features stand out: the linear probe needs many more epochs (28) to converge than any unfrozen variant (9), because only 3,078 parameters carry the entire adaptation; and once `layer4` is unfrozen, adding roughly 2.8M further trainable parameters (`layer3` through the full network) leaves test accuracy statistically unchanged. The chosen partition therefore sits exactly at the point of diminishing returns.

H. First-Layer Substitution with Gabor Filters

The freeze-depth sweep asks how much of the pretrained network must adapt when the network is otherwise fixed.

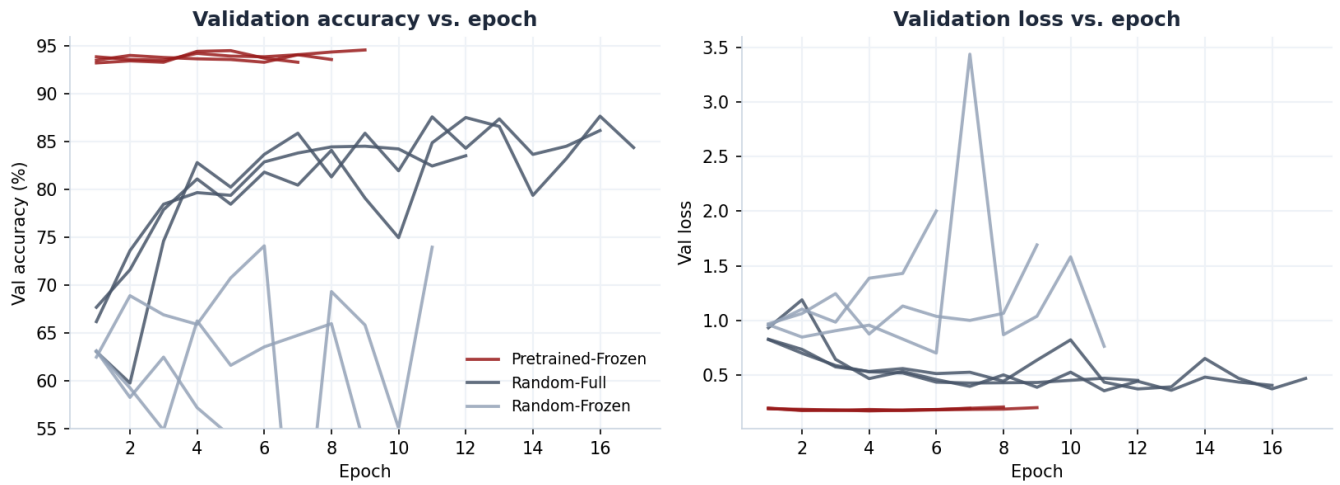


Fig. 1. Per-epoch validation accuracy (left) and loss (right) for all three conditions across the three seeds. Pretrained-frozen (dark red) converges within a few epochs; random-full (slate) climbs slowly over many epochs; random-frozen (light) is noisy and plateaus low.

PCA of penultimate (512-d) features on the test split · seed 42

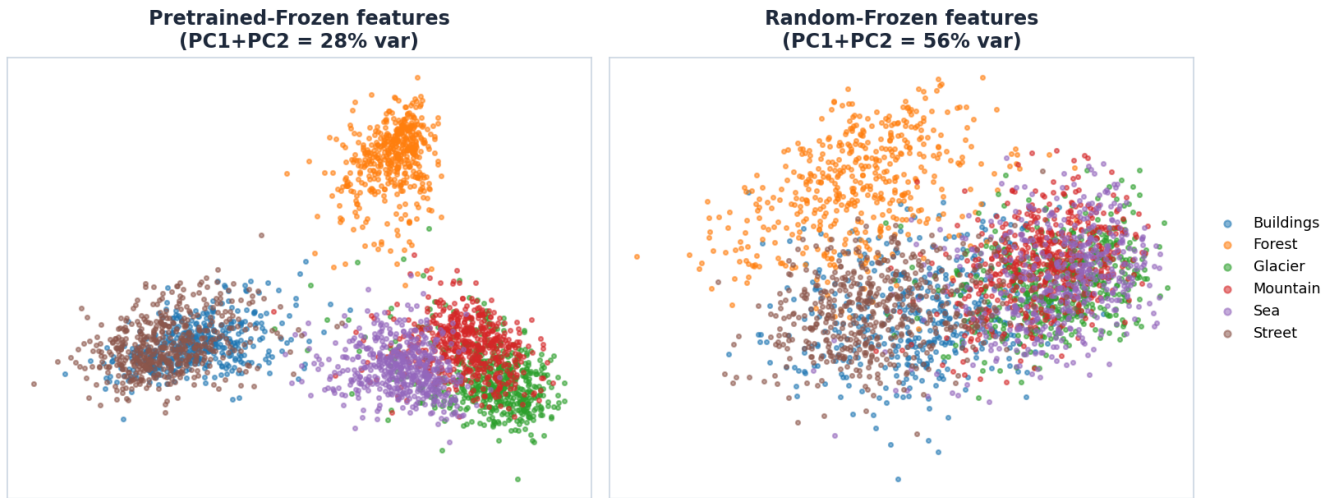


Fig. 2. PCA of penultimate (512-d) test features (seed 42). Pretrained-frozen features (left) separate the six classes; random-frozen features (right) remain entangled, with only forest partially separable.

A sharper version of that question targets the *first* layer specifically: if the learned first convolution is replaced by a fixed, hand-designed front end, how far up the network must weights re-adapt to compensate, and how much accuracy is permanently lost? Gabor filters are the classic hand-designed front end, long used as oriented edge and texture detectors [11], and fixed wavelet front ends have been shown to rival learned first layers on some recognition tasks [12].

I replace ResNet-18’s first convolution (`conv1`) with a fixed bank of 64 Gabor filters spanning 8 orientations, 4 wavelengths, and 2 phases (even and odd), and freeze it. Every other layer starts from ImageNet-pretrained weights. I then run the same freeze-depth sweep, unfreezing the residual stages as a growing prefix from the input side (`fc` read-out, then

`+layer1`, `+layer2`, `+layer3`, and finally `+layer4`, i.e. everything except the frozen first conv), on the full training set at seed 42. As a matched reference I run the identical sweep with the ordinary learned `conv1` (also frozen). `bn1` and the head are trainable throughout so the fixed front end can be re-normalized and read out.

Figure 4 shows both curves. With the learned first layer, a read-out already reaches 91.6% and unfreezing more stages rises gently to 93.2%. The Gabor front end behaves very differently. At read-out it scores only 81.1%, roughly 10 points below the learned first layer, since a fixed Gabor `conv1` produces activations the pretrained upper layers were never built for. Unfreezing the single next stage (`layer1`) recovers most of that gap, jumping to 86.1% (+5.0 points). After

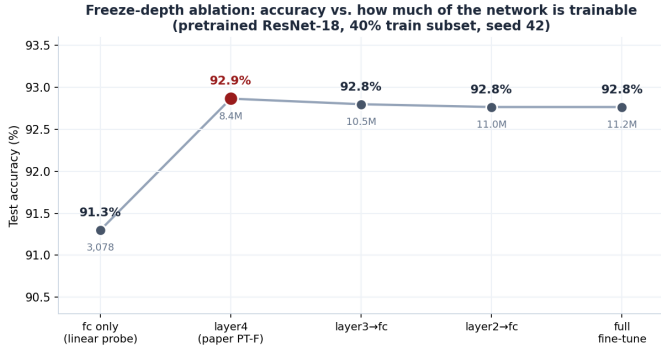


Fig. 3. Test accuracy as the pretrained network is progressively unfrozen. Most of the benefit is already reached by a linear probe; training `layer4` (the boundary used in the main experiments) captures the rest; deeper fine-tuning adds nothing.

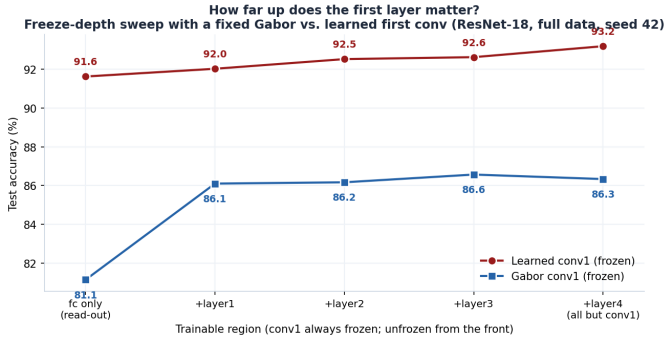


Fig. 4. Freeze-depth sweep with a fixed Gabor first layer versus the learned first layer (both frozen; ResNet-18, full data, seed 42). Swapping in Gabor filters costs about 10 points at read-out; re-adapting only `layer1` recovers most of it, and unfreezing deeper stages adds almost nothing, so the first-layer influence reaches only about one stage up. A gap of roughly 7 points to the learned first layer persists even at full trainability.

that the curve is essentially flat: `+layer2`, `+layer3`, and `+layer4` give 86.2%, 86.6%, and 86.3%, so unfreezing anything beyond `layer1` makes almost no difference.

Two conclusions follow. First, the disturbance introduced by swapping the first layer is absorbed almost entirely by re-adapting the immediately following stage; its influence does not propagate more than about one stage up the network, because letting deeper stages adapt buys nothing. Second, the substitution leaves a persistent gap of roughly 7 points (86.3% versus 93.2%) even when every layer above `conv1` is trainable and the two runs have identical trainable capacity. The learned first-layer filters therefore carry information that a fixed Gabor bank plus unrestricted downstream retraining cannot reconstruct, a concrete measure of how much the first layer alone contributes on this task.

V. DISCUSSION AND SUMMARY

A. Isolating Pretraining Quality

PT-F and R-F share the same trainable parameters and the same freeze boundary, so the gap between them is down to feature quality alone. Table I shows that gap is large: over 50

points of test accuracy on average. Since θ_f in R-F is random and never updated, `layer4` has to turn essentially arbitrary patterns into class scores, which caps how far it can generalize. Even so, R-F reaches 66.5%, well above the 16.7% chance rate for six classes. That fits the finding of Saxe et al. [13] that random convolutional filters still carry some frequency- and orientation-selective structure through pooling, so the random frozen extractor is a real baseline, not pure noise. PT-F, by contrast, starts from ImageNet weights that already encode edges, textures, and shape boundaries, and those features carry over well to the natural scenes in the Intel set.

B. Isolating Trainable Capacity

R-F and R-FL both start from random weights, so the gap between them is about capacity. R-FL can adapt every parameter, including θ_f , so its early layers learn filters from the pixels up to fit the data instead of being stuck at their initial values. That lifts its mean accuracy to 84.69%, against the restricted 66.54% of R-F.

R-FL also takes much longer to train, converging in a mean of 15.0 epochs against 8.7 for R-F, and it still trails PT-F by a wide margin. So extra capacity helps, but it cannot make up for a poor starting point when the data and the epoch budget are limited.

C. Class-wise Confusions

My test confusion matrices show consistent patterns of category-specific confusion across all conditions, though the errors are far more pronounced in the random initialization models. The primary confusion occurs between:

- **Glacier vs. Mountain:** Visual features such as white snow, jagged ridges, and rock textures are shared heavily between these classes.
- **Buildings vs. Street:** Images of streets frequently contain buildings in the background, and vice-versa, making the spatial context highly overlapping.

These errors come from genuine visual overlap in the data, not from anything in the training setup.

D. Limitations and Future Work

A few things limit these conclusions. I use one target dataset and one backbone (ResNet-18), so the size of the pretraining benefit could look different on other domains or deeper networks. The freeze-depth ablation of Section IV-G ran on a 40% training subset at a single seed to save compute, and the Gabor substitution of Section IV-H reports a single seed on full data; both trends are clear, but tighter error bars would need several seeds. Every condition also shares one optimizer, learning rate, and early-stopping patience, so the comparison reflects a typical low-tuning setting rather than each method at its best.

The most useful next step is a data-fraction sweep. He et al. [4] argue that from-scratch training can match pretraining given enough data and iterations, so scaling the labeled set from a few hundred images up to the full split would show where, if anywhere, the two meet on scene classification.

Running the ablation and the main comparison on a second backbone and a second dataset would show how far the “most of the value needs almost no fine-tuning” result carries.

E. Summary

The goal was to separate two things that a plain pretrained-versus-scratch comparison mixes together, and the capacity-matched control did that. With the trainable parameters held fixed, pretrained features are worth more than 50 points of test accuracy over random ones, and they converge in far fewer epochs. The supporting views agree: the learning curves show pretraining is useful almost from the first epoch rather than just ending higher, and the PCA shows why, with pretrained features falling into class-separable clusters while random frozen features stay mixed. The freeze-depth sweep adds a practical note, since a bare linear probe already recovers most of the accuracy, so on this task the benefit of pretraining needs almost no fine-tuning to reach. When data or compute is limited, starting from pretrained weights is the clear choice.

REFERENCES

- [1] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [3] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 3320–3328.
- [4] K. He, R. Girshick, and P. Dollár, “Rethinking imagenet pre-training,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 4918–4927.
- [5] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell, “Decaf: A deep convolutional activation feature for generic visual recognition,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2014, pp. 647–655.
- [6] A. Sharif Razavian, H. Azizpour, J. Sullivan, and S. Carlsson, “Cnn features off-the-shelf: an astounding baseline for recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2014, pp. 806–813.
- [7] S. Kornblith, J. Shlens, and Q. V. Le, “Do better imagenet models transfer better?” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2661–2671.
- [8] B. Zhou, A. Lapedriza, J. Xiao, A. Torralba, and A. Oliva, “Learning deep features for scene recognition using places database,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 487–495.
- [9] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
- [10] Intel Corporation and P. Bansal, “Intel image classification dataset,” <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>, 2019.
- [11] A. K. Jain and F. Farrokhnia, “Unsupervised texture segmentation using gabor filters,” *Pattern Recognition*, vol. 24, no. 12, pp. 1167–1186, 1991.
- [12] J. Bruna and S. Mallat, “Invariant scattering convolution networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 35, no. 8, pp. 1872–1886, 2013.
- [13] A. M. Saxe, P. W. Koh, Z. Chen, M. Bhand, B. Suresh, and A. Y. Ng, “On random weights and unsupervised feature learning,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2011, pp. 1089–1096.